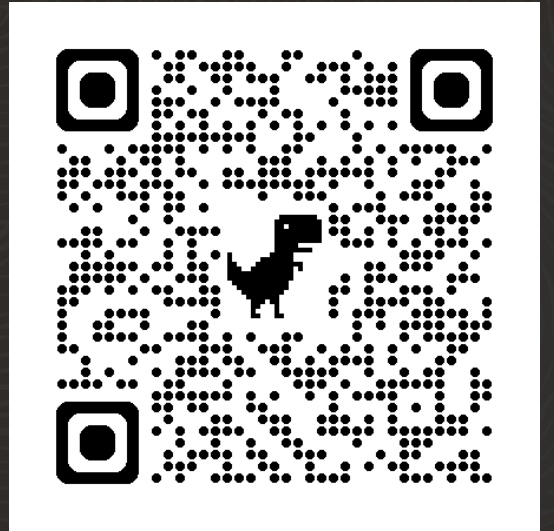




<https://github.com/albertattard/demo>

Examples



Java in Containers

Build faster, smaller and more secure Java containers

Albert Attard

Principal Java Success Consultant

Java Success Consulting Team

July 2024



Who am I?

Who am I?



- Albert Attard
- Born in Malta (a long time ago)
- Passionate Java Developer
- Moved to Köln, Germany, in Oct. 2018
- Joined Oracle in July 2022, as part of the Java Success Consultancy team

Who am I?

Where is Malta?



What do you expect to gain from this session about Java in containers?

Objectives

- Configure and diagnose Java applications running in containers
- Build smaller, efficient and secure container images
- Create portable, multi-architecture images for Java applications



Mastery takes about 10,000 hours...

Good news: We're not doing all that today.

You are not expected to become an expert after this workshop

but rather to simply identify gaps in your knowledge and begin a conversation

*“If there’s something strange, in your **Java** code, who you gonna call?”*

Java Success Team!



References

Ops.java: The Destination for Java Administrators

ops.java

Java™

Download

Developer Resources

Report Issue

Article search...

The Destination for Java Administrators

Current, Up-to-Date Releases	26.0.1	25.0.3	21.0.11	17.0.19	11.0.31	8u491
Next Critical Patch Update (2026-07-21)	26.0.2	25.0.4	21.0.12	17.0.20	11.0.32	8u501

Guides



Security

Cryptographic algorithms, trust management, X.509 certificates, vulnerability protection.



Troubleshooting

Guides and background information for troubleshooting problems.



Performance

Garbage collection and memory, troubleshooting performance issues and exceptions, performance tuning.



Serviceability Tools

Diagnostic and troubleshooting tools such as Java Mission Control, Flight Recorder, jcmd, and more.



How-To

Guides for performing a variety of tasks.



Java Support Handbook

Learn how to leverage the benefits of your Java SE Subscription.



Workshops on LiveLabs

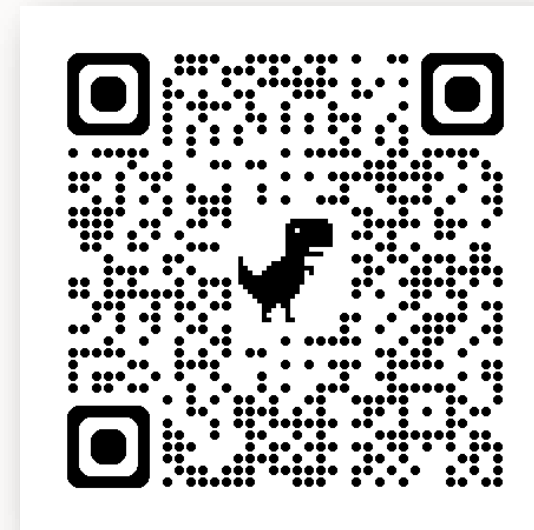
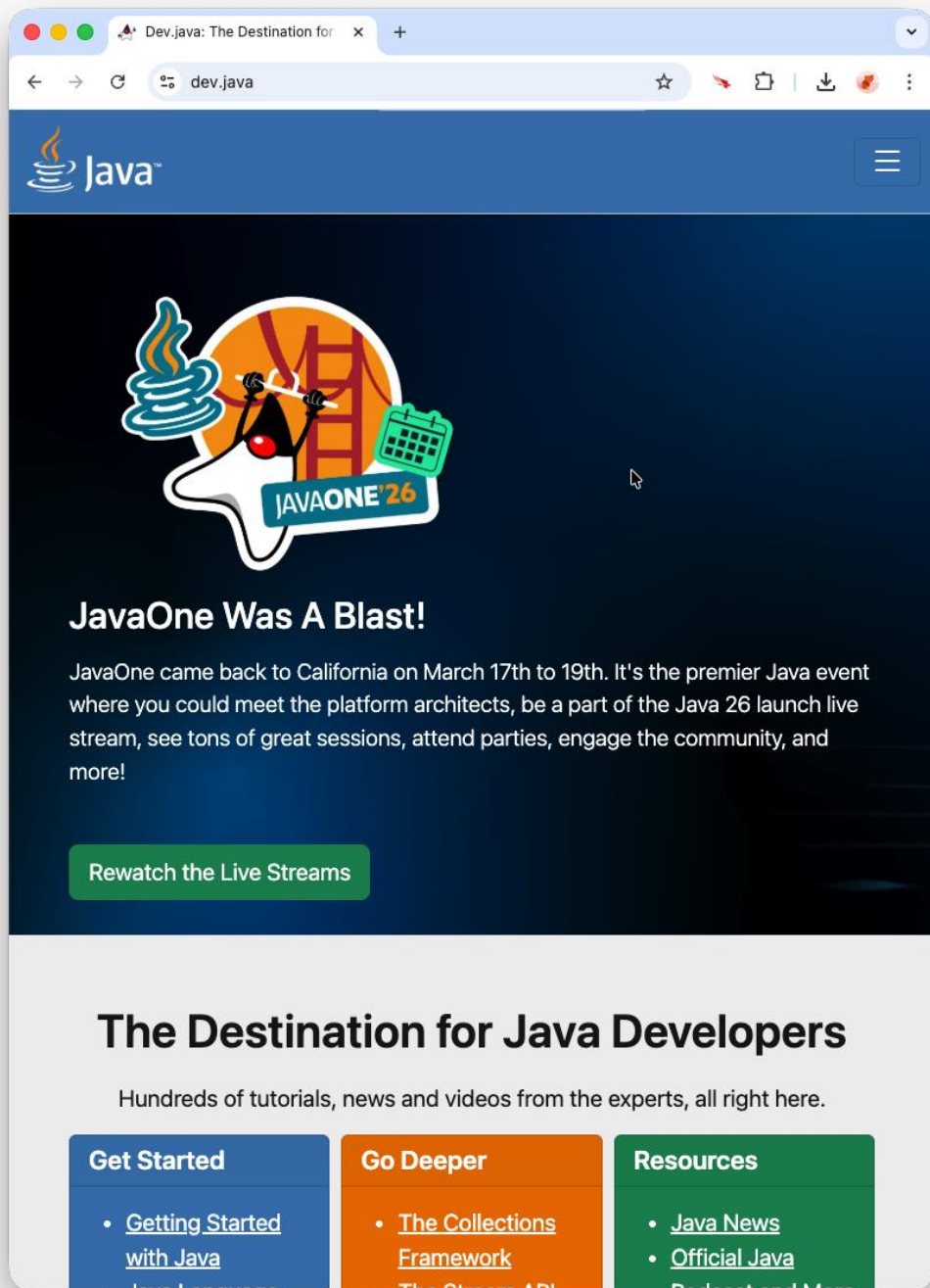
Workshops targeted at system administrators on how to use...

ops.java

Oracle

12 Copyright © 2026, Oracle and/or its affiliates | albert.attard@oracle.com

03.07.2026



dev.java

Oracle



Performance – Inside.java x +

inside.java/tag/performance

 **Inside Java**
News and views from members of the Java team at Oracle

[Newscast](#) | [Podcast](#) | [JEP Café](#) | [Sip of Java](#) [dev.java](#) | [Newsletter](#) | [About](#)

Sort by: [Date](#) | [Author](#) | [Tag](#)

JDK 27 JDK 26 JDK 25 JDK 24 JDK 23 JDK 22 JDK 21 JDK 20
JDK 19 JDK 18 JDK 17 JDK 16 JDK 15 JDK 14 JDK 13 JDK 12 AI
Amber Babylon Client Cloud Community Concurrency Core Libraries
Education GC HotSpot Integrity by Default JFR Java Language Leyden
Loom Modules Moved By Java Networking Oracle Panama **Performance**
Security Serviceability Valhalla

[Clear Selection](#)

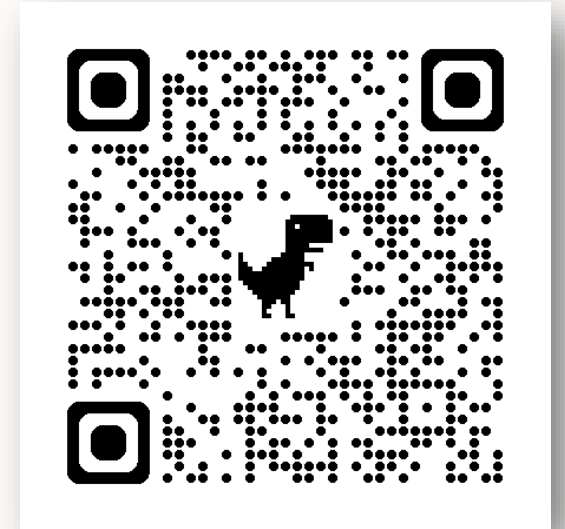
Performance

2026

- [Secure, Scalable JVM Diagnostics for Kubernetes with JMS](#) by [Praveen Srivastava](#) on March 13, 2026
- [Java Performance Update: From JDK 21 to JDK 25](#) by [Claes Redestad](#), [Per-Ake Minborg](#) on March 8, 2026
- [Java 26 for DevOps](#) by [Billy Korando](#) on March 2, 2026
- [1B Rows with the Memory API - JEP Cafe #25](#) by [José Paumard](#) on January 17, 2026
- [One Giant Leap: 95% Less Sampling Cost](#) by [Jonas Norlinder](#) on January 14, 2026
- [The Static Dynamic JVM – A Many Layered Dive #JVMLS](#) by [John Rose](#) on January 11, 2026
- [Run Into the New Year with Java's Ahead-of-Time Cache Optimizations](#) by [Ana-Maria Mihalceanu](#) on January 9, 2026

2025

- [Supercharge your JVM Performance with Project Leyden and Spring Boot](#) by [Ana-Maria Mihalceanu](#) & [Moritz Halbritter](#) (guest) on November 2, 2025
- [Assembling Project Leyden #JVMLS](#) by [Dan Heidinga](#) on October 21, 2025



inside.java

Oracle



Command Line Arguments

Command Line Arguments

- Command line arguments can be passed to both the JVM and to the application

```
java [JVM launcher options] -jar application.jar [application args]
```

Command Line Arguments

Environment Variables

- `_JAVA_OPTIONS`: Global forced options
- Command line arguments
- `JDK_JAVA_OPTIONS`: Defaults for `java`
- `JAVA_TOOL_OPTIONS`: JVM-wide tool injection
- `JAVA_OPTS`: Script-only convention

Command Line Arguments

- **JDK_JAVA_OPTIONS: Prepends default options to the `java` launcher command line**

```
JDK_JAVA_OPTIONS="-Ddemo.env=workshop -XX:MaxRAMPercentage=75"
```

Command Line Arguments

- **JAVA_TOOL_OPTIONS: Injects JVM options when the VM is created, mainly for tools, agents, or indirect launches.**

```
JAVA_TOOL_OPTIONS="-javaagent:/opt/agent.jar -Dmonitoring.enabled=true"
```

Command Line Arguments

Argument files: @filename

- @filename expands JVM launcher arguments from a file at that exact command-line position

```
java @jvm.options -jar application.jar [application args]
```

jvm.options

```
-Xmx64m  
-Ddemo.source=argument-file
```

Default Maximum Heap Size

- What is the default maximum heap size in Java when not explicitly configured, and how is it determined based on the container's memory limit?
- Java allocates a maximum heap size equal to **25%** of the container's memory limit.

Print VM Settings

- The `-XshowSettings:vm` prints the maximum heap size together with other information

```
> java \
  -XshowSettings:vm \
  --version

VM settings:
  Max. Heap Size (Estimated): 7.77G
  Using VM: Java HotSpot(TM) 64-Bit Server VM
```

Print VM Settings

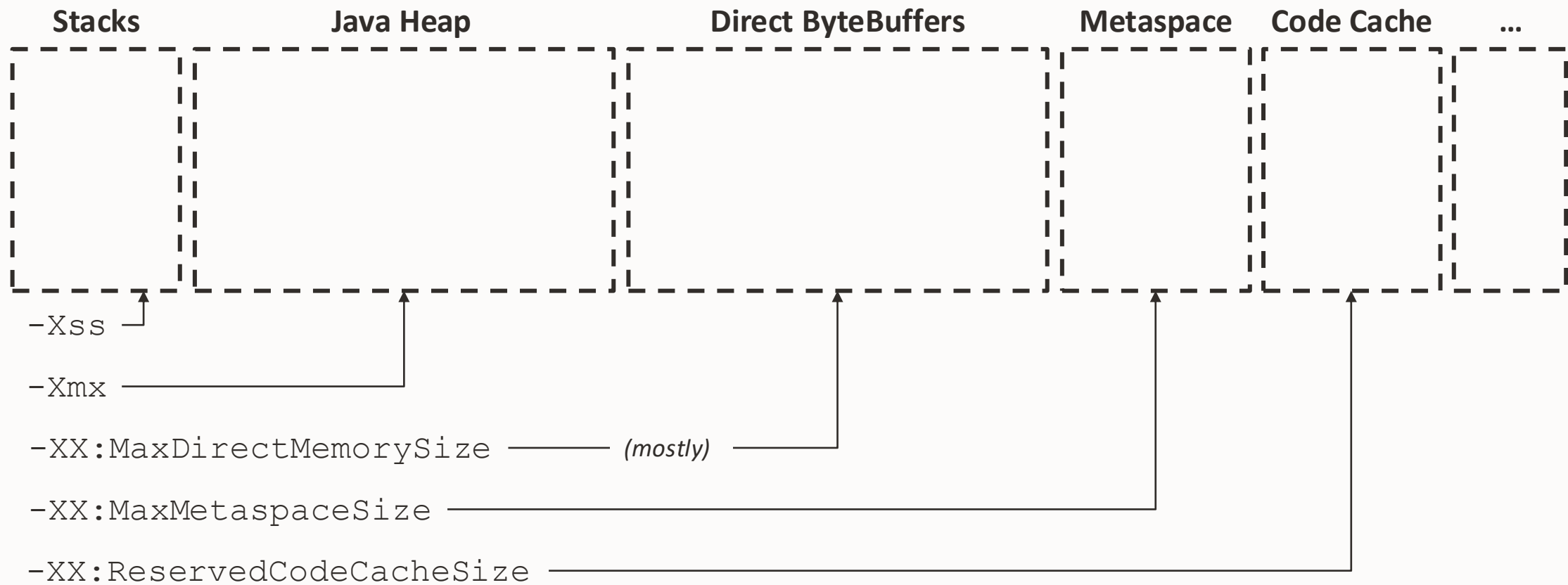
- The `-XX:+PrintFlagsFinal` Java command-line option displays the final values of all JVM flags, including their types, origins, and current settings, after any modifications from defaults, environment variables, or command-line arguments

Print VM Settings

```
> java \
  -XX:+PrintFlagsFinal \
  --version \
  | grep -Ei 'maxheapsize|maxram'

size_t MaxHeapSize      = 8346664960      {product} {ergonomic}
uint64_t MaxRAM          = 137438953472    {pd product} {default}
uintx MaxRAMFraction    = 4                {product} {default}
double MaxRAMPercentage = 25.000000       {product} {default}
size_t SoftMaxHeapSize   = 8346664960      {manageable} {ergonomic}
```

Memory



Memory management

- Set the maximum amount irrespective of the available memory

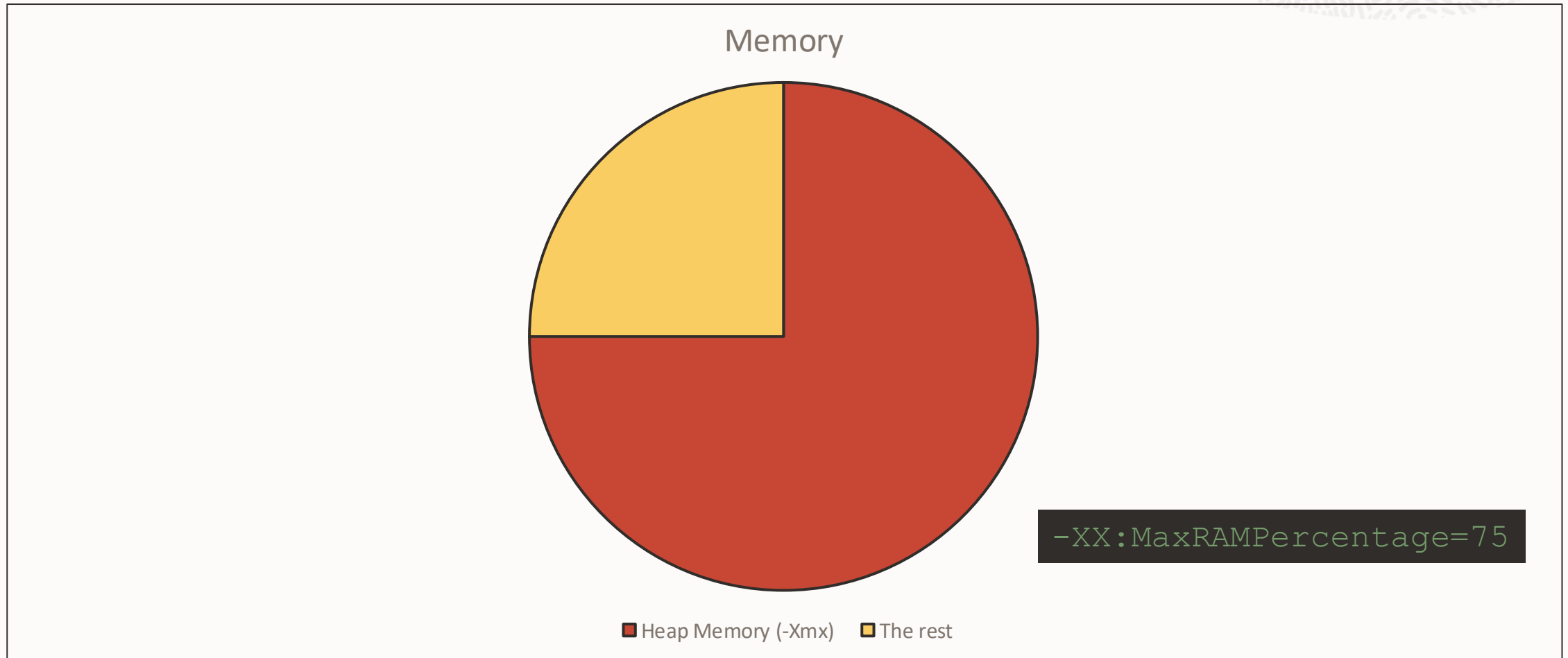
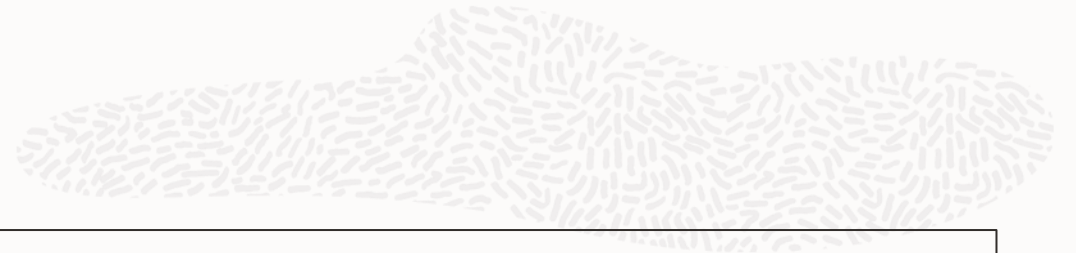
```
> java -Xmx### ...
```

- Alternatively, set the maximum memory relative to the available memory

```
> java -XX:MaxRAMPercentage=## ...
```

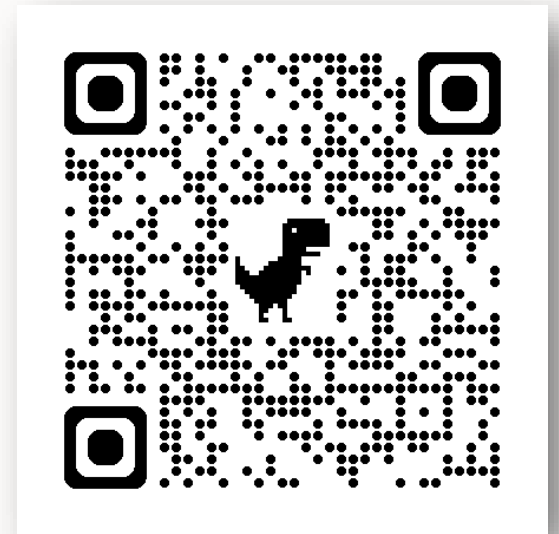
Memory management

A starting point



Memory management

- More information about the available options can be found [here](#)



Demo

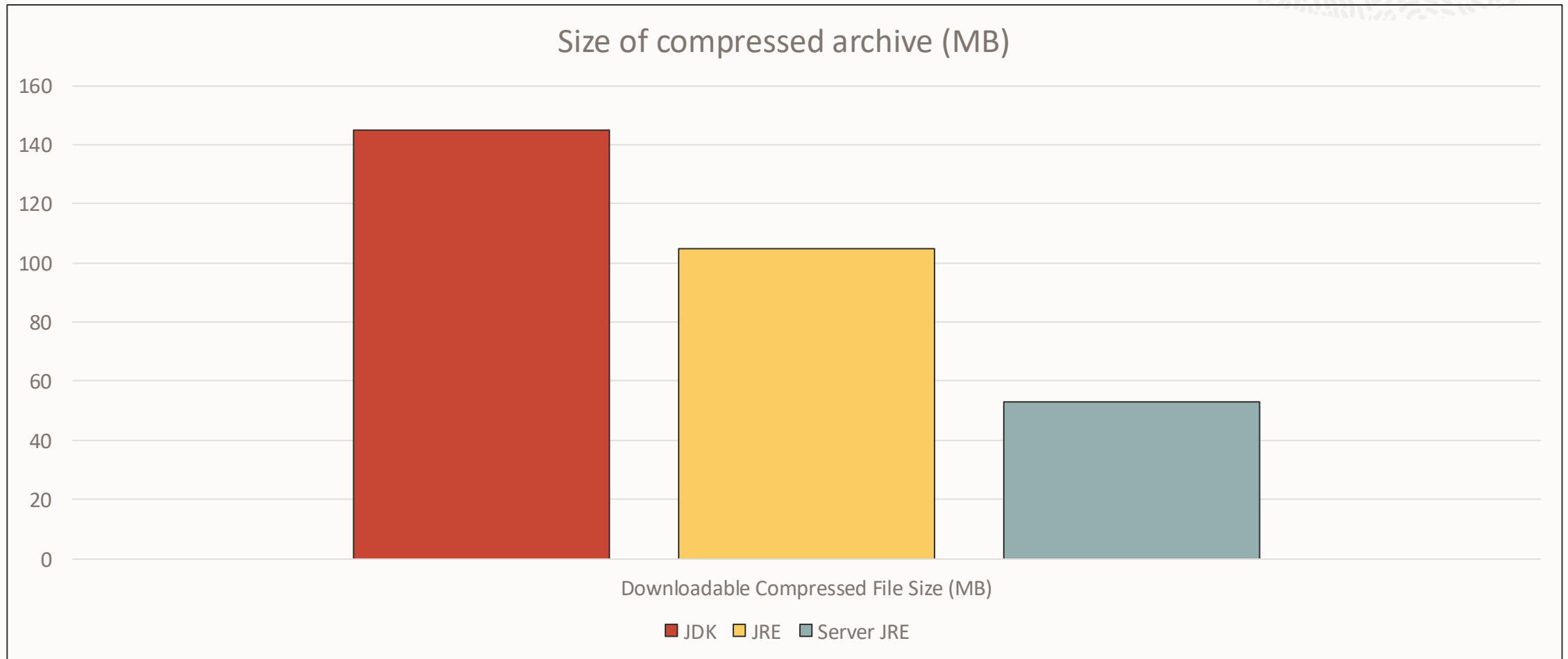
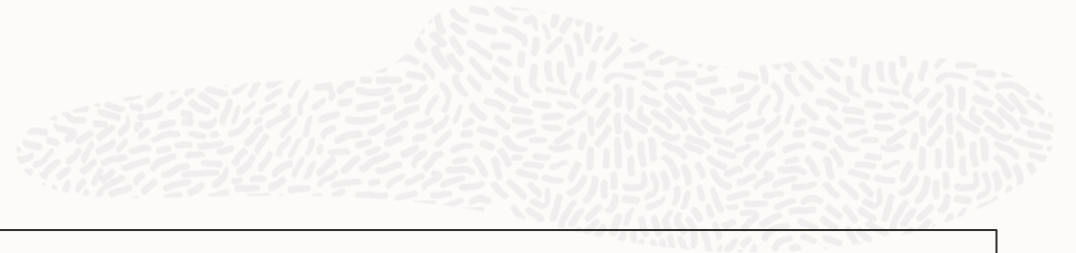
`java-command-line-options-spring-boot`

Custom runtime images

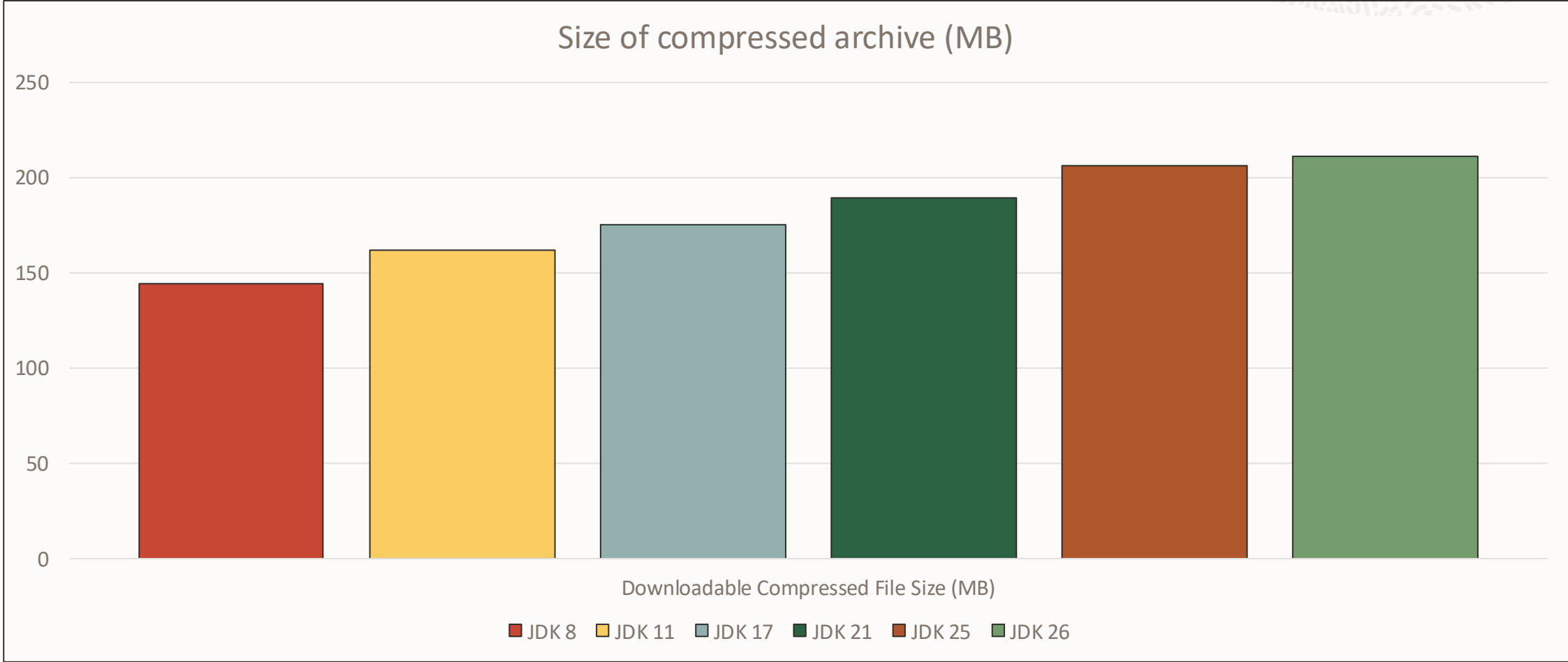
Java 8 distributions

- Java SE Development Kit (JDK)
- Java SE Runtime Environment (JRE)
- Server Java SE Runtime Environment (Server JRE)

Java 8 distributions



JDK size



Java Distributions

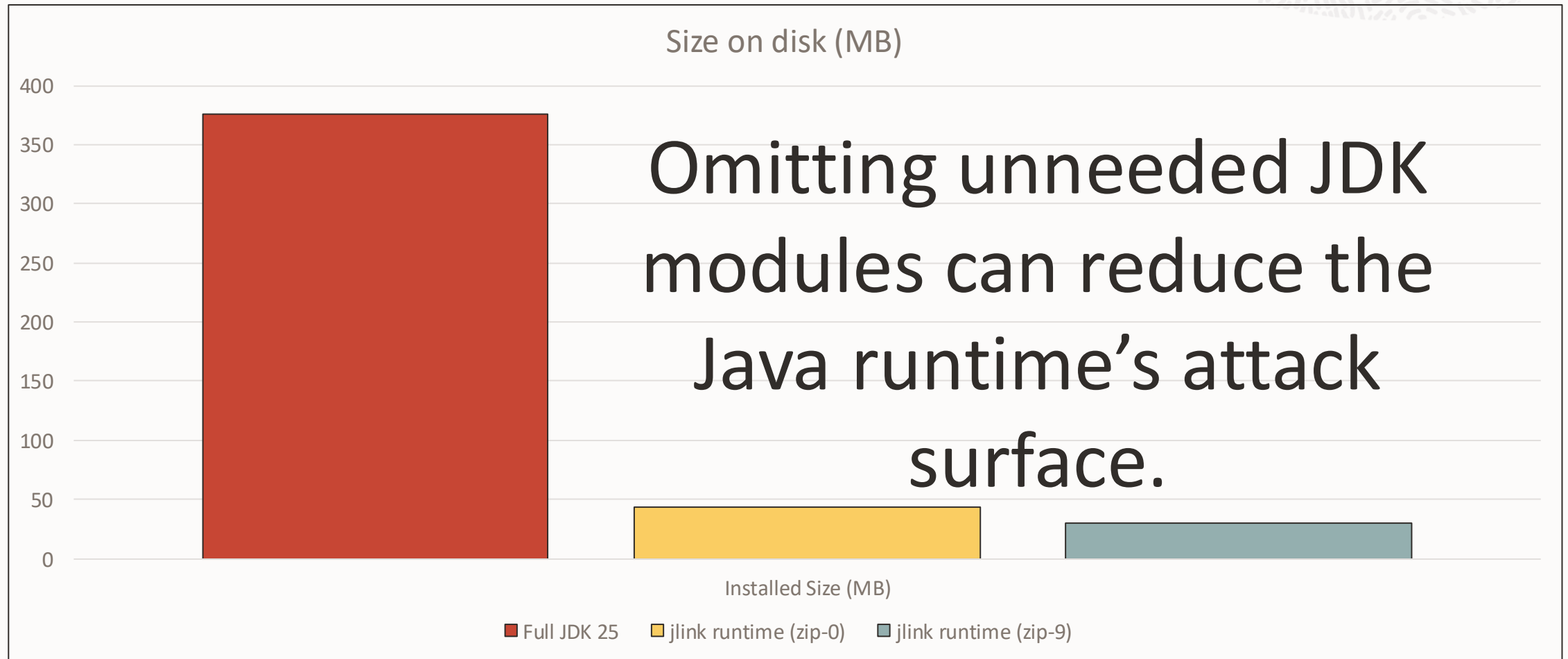
- Java 8 and earlier: the JDK included a `jre/` directory, and a separate JRE distribution was available.
- Java 9: the JDK and JRE became modular runtime images; `jlink` was introduced.

Java Distributions

- Java 9 onwards: Oracle no longer provides separate JRE downloads.
- Use `jlink` to create an application-specific runtime image containing only the required modules and dependencies.

Minimal jlink runtime

Runtime image size: `java.base` only



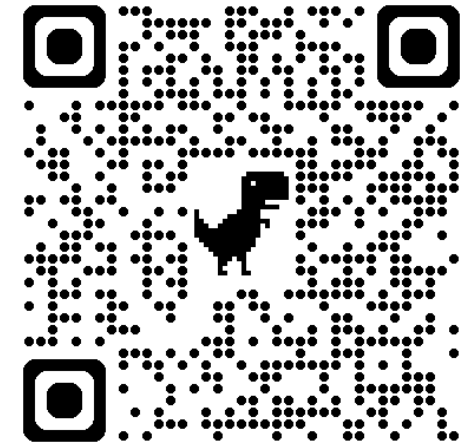
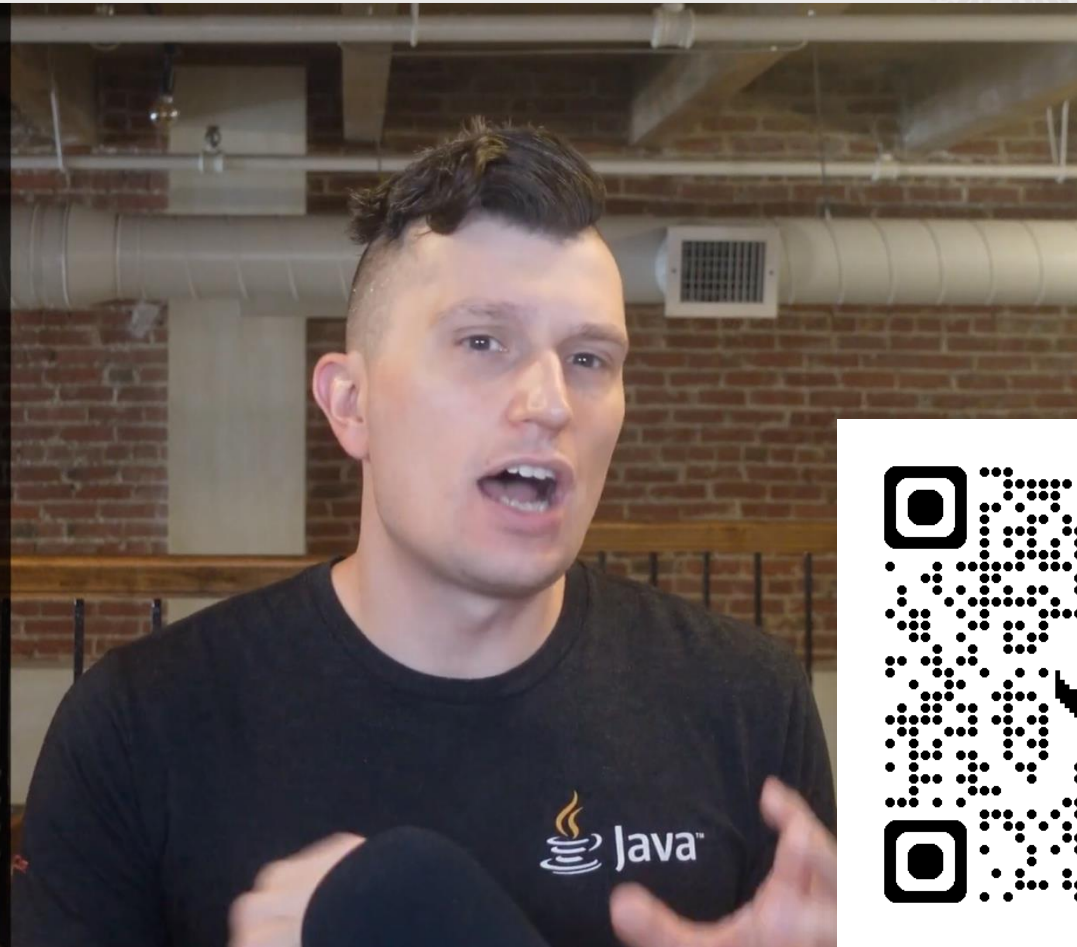
Minimal jlink runtime

```
> jlink \  
  --output './custom-image' \  
  --no-header-files \  
  --no-man-pages \  
  --add-modules java.base,... \  
  --strip-debug \  
  --compress=zip-9
```

How-to Build Custom Java Runtimes with Jlink

Agenda

1. Basics of creating a runtime image
2. Using JDepends to find modules
3. Adding non-JDK modules to an image
4. Creating and adding your own modules
5. Additional image customization options



Demo

`jlink-slim-image-hello-world`

Resolving dependencies

- Applications usually require more than `java.base`.
- Use `jdeps` to identify the candidate JDK modules required by an application JAR.

Resolving dependencies

```
> jdeps \  
--multi-release 25 \  
--ignore-missing-deps \  
--print-module-deps \  
'application.jar'
```

Resolving dependencies

- Spring Boot executable JARs store application classes and dependencies as nested JARs.
- Analysing the outer JAR with `jdeps` alone can produce an incomplete module list.

Resolving dependencies

- Provide the dependencies on `jdeps`' class path, then verify the runtime with application and integration tests.

Resolving dependencies

```
> jdeps \  
  --class-path '...' \  
  --multi-release 25 \  
  --ignore-missing-deps \  
  --print-module-deps \  
  'application.jar'
```

Demo

`jlink-slim-image-spring-boot`

Diagnostics

Diagnostics are a packaging requirement

- Before minimising a Java runtime, define the incident-response workflow.
- `jcmd` and JFR depend on what the image contains.

Diagnostics are a packaging requirement

- Native Memory Tracking (NMT) must be enabled when the JVM starts, and inspecting it with `jcmd` also requires the diagnostic tooling.
- Live Native Memory Tracking requires a diagnostic interface, such as `jcmd` or secured JMX.

The operational trade-off



Runtime	Serves HTTP	jcmd/JFR	NMT
Full JDK	Yes	Available	Available if enabled at startup
Minimal <code>jdk-link</code> runtime	Yes	Not available	Cannot be inspected with <code>jcmd</code>
Diagnostic <code>jdk-link</code> runtime	Yes	Available	Available if enabled at startup

A smaller image is not the sole optimisation target: removing `jdk.jcmd` and `jdk.jfr` removes diagnostic capability.

Demo

container-diagnostics-spring-boot

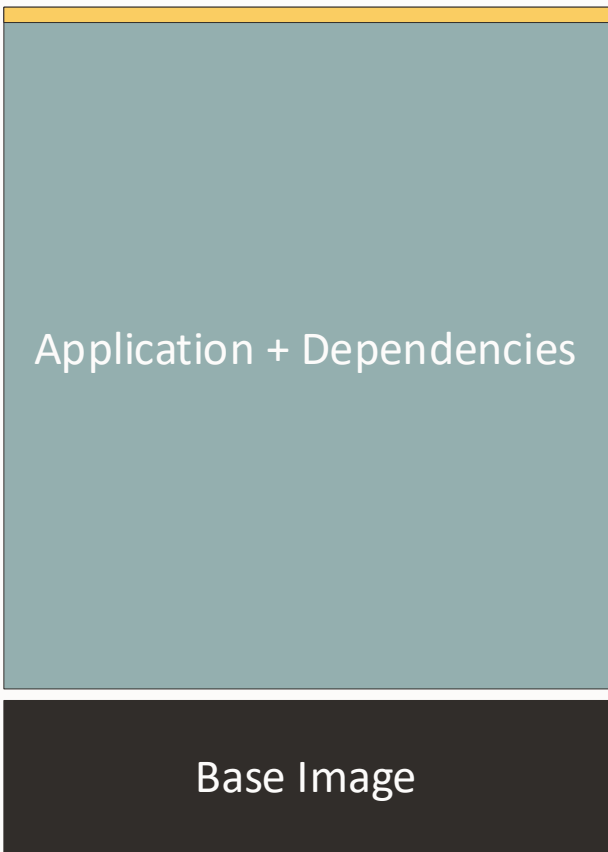


Layered JARs

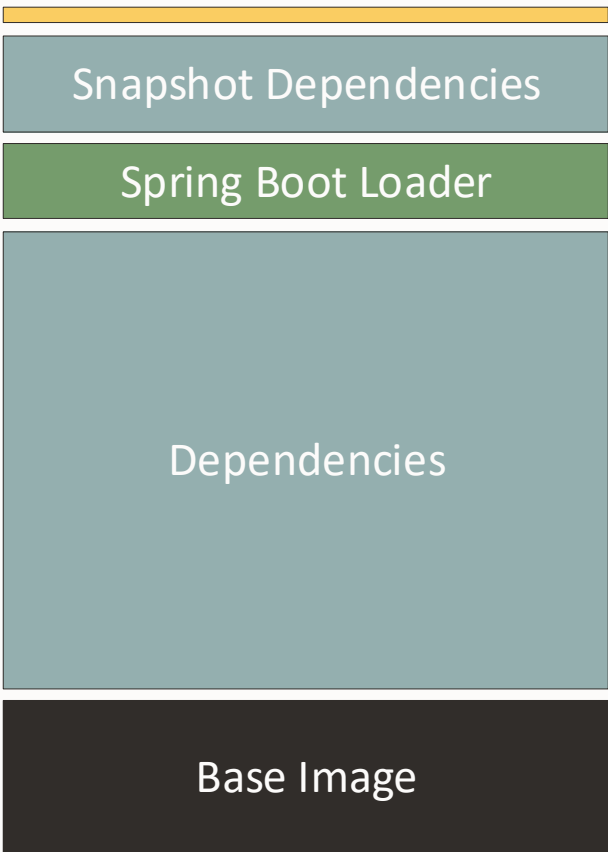
Container Layers

- Avoid copying unchanged dependencies when only application code changes.
- Separate stable dependencies from application code to reuse cached layers and speed up rebuilds and pushes.

Container Layers



Application

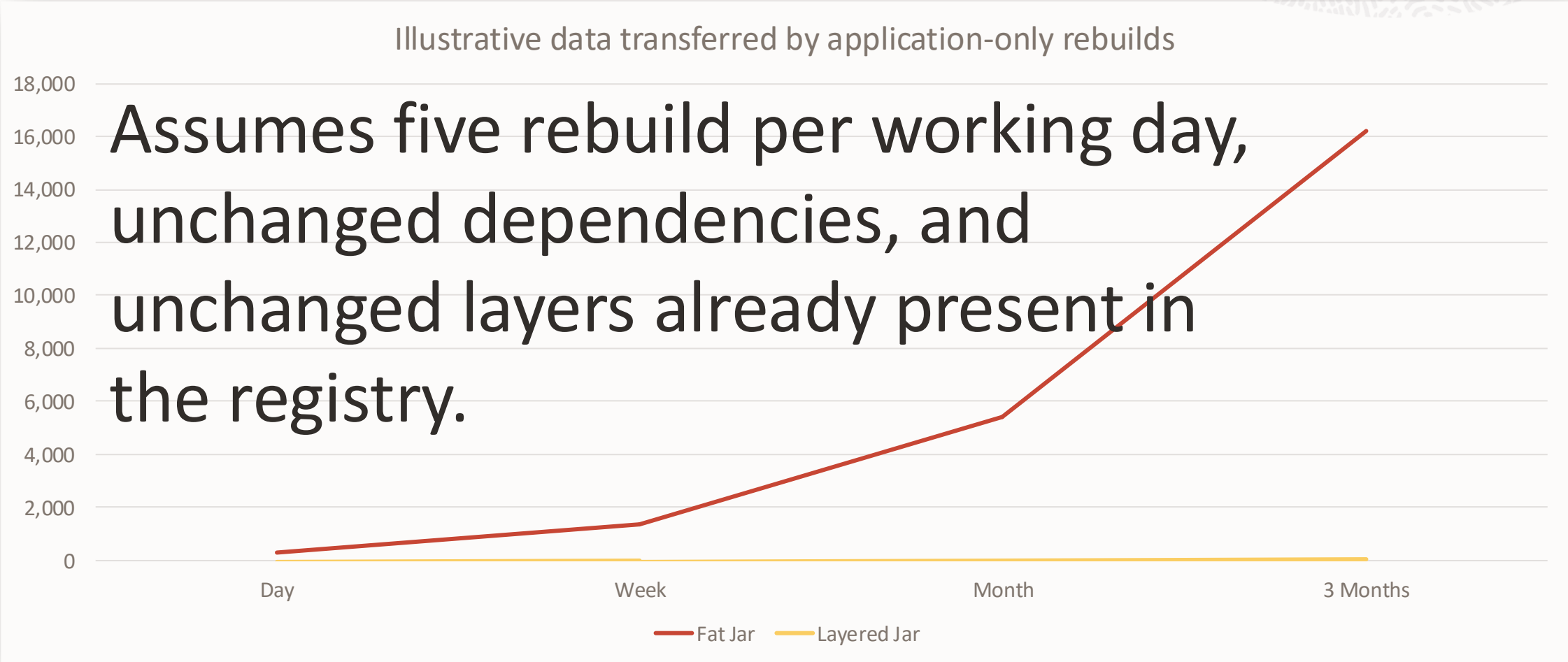


Container Layers



- Layer order matters: copy the least frequently changing content first and the application layer last.
- This optimises build and distribution efficiency; it does not inherently reduce image size or improve application runtime performance.

Container Layers



Demo

layered-jar-spring-boot

Container Image Hygiene Checks

Container Image Hygiene Checks



- Make selected container-image decisions visible and reviewable.
- Each check addresses a different risk.
- Together, they provide evidence, **not a security guarantee.**

Application user

- Many applications run as root in their container without an explicit decision.
- Run the process as a non-root numeric user

```
COPY --chown=10001:0 ./target/application-*.jar app.jar  
USER 10001:0
```

Reproducibility

- Container tags are mutable
- Pin the base image to the digest to ensure reproducibility

```
FROM container-  
registry.oracle.com/java/jdk:25.0.3@sha256:8d233560cfa  
91301d256148ecb6f031b3c96a77bf4153dad8bae65b4ecb19bb9
```

Container image SBOM

- A simple Java container image can contain thousands of components from the application and its base image

```
> syft 'docker-archive:target/application.tar' \  
  --output 'cyclonedx-json=target/sbom.cdx.json'
```

Vulnerability scanning



- Report high and critical known vulnerabilities that have an available fix

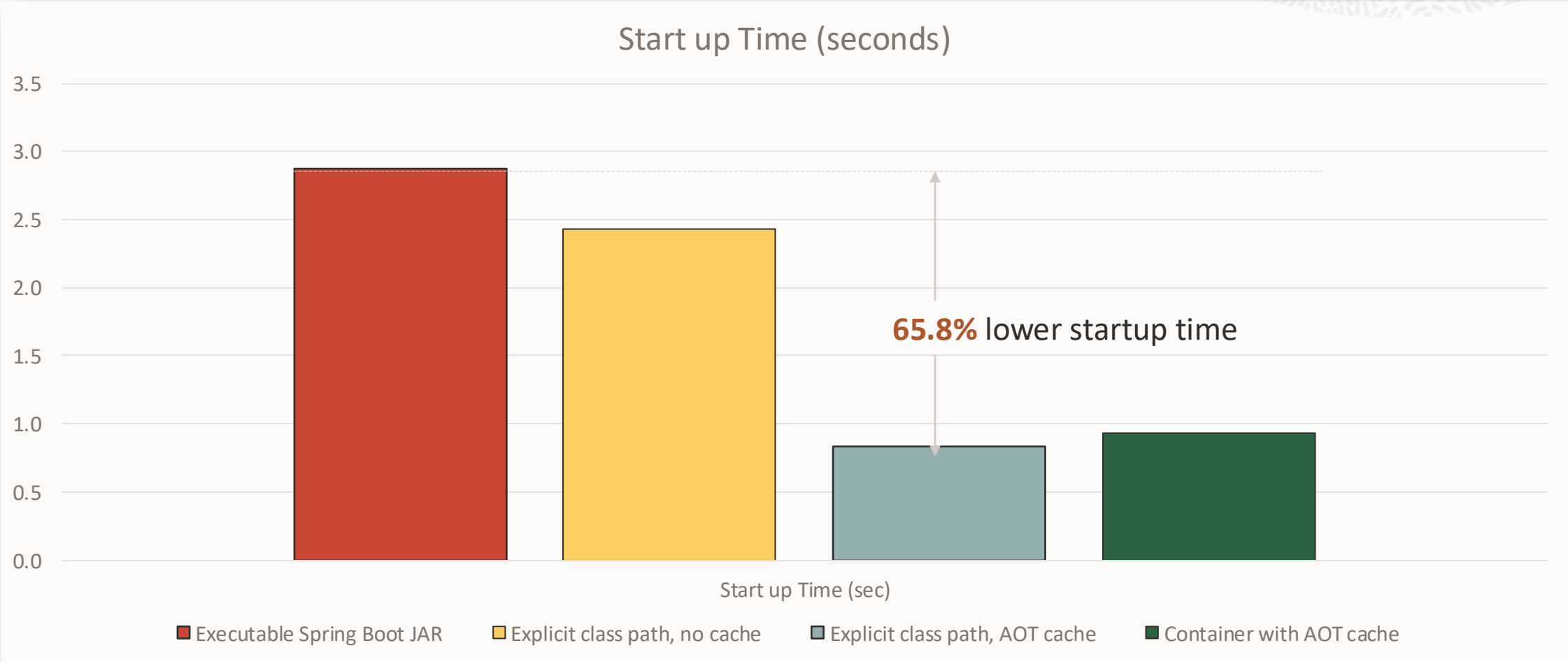
```
> trivy image \  
  --severity HIGH,CRITICAL \  
  --ignore-unfixed \  
  --input 'target/application.tar'
```

Demo

container-image-hygiene-spring-boot

Project Leyden: AOT cache

A local startup comparison



AOT Cache



- Create the AOT cache

```
> java \  
-Dspring.context.exit=onRefresh \  
-XX:AOTCacheOutput='./target/app.aot' \  
@./target/java.args
```

- Use the AOT cache

```
> java \  
-XX:AOTCache='./target/app.aot' \  
@./target/java.args
```

The same `java` arguments are used to create the AOT cache and run the application with the AOT cache.

Demo

leyden-aot-cache-spring-boot

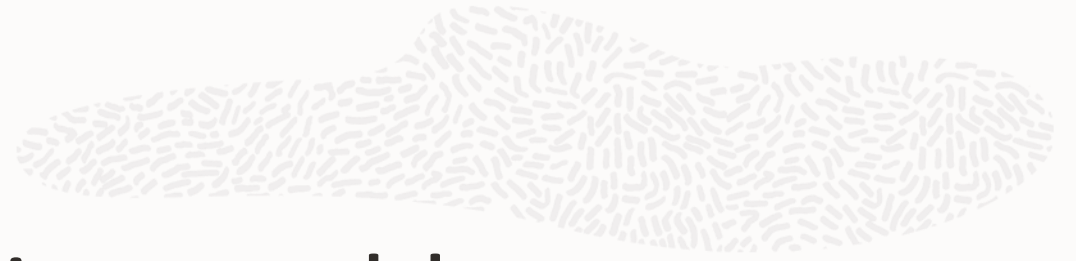
Multi-Architecture Container Images

Multi-Architecture Container Images



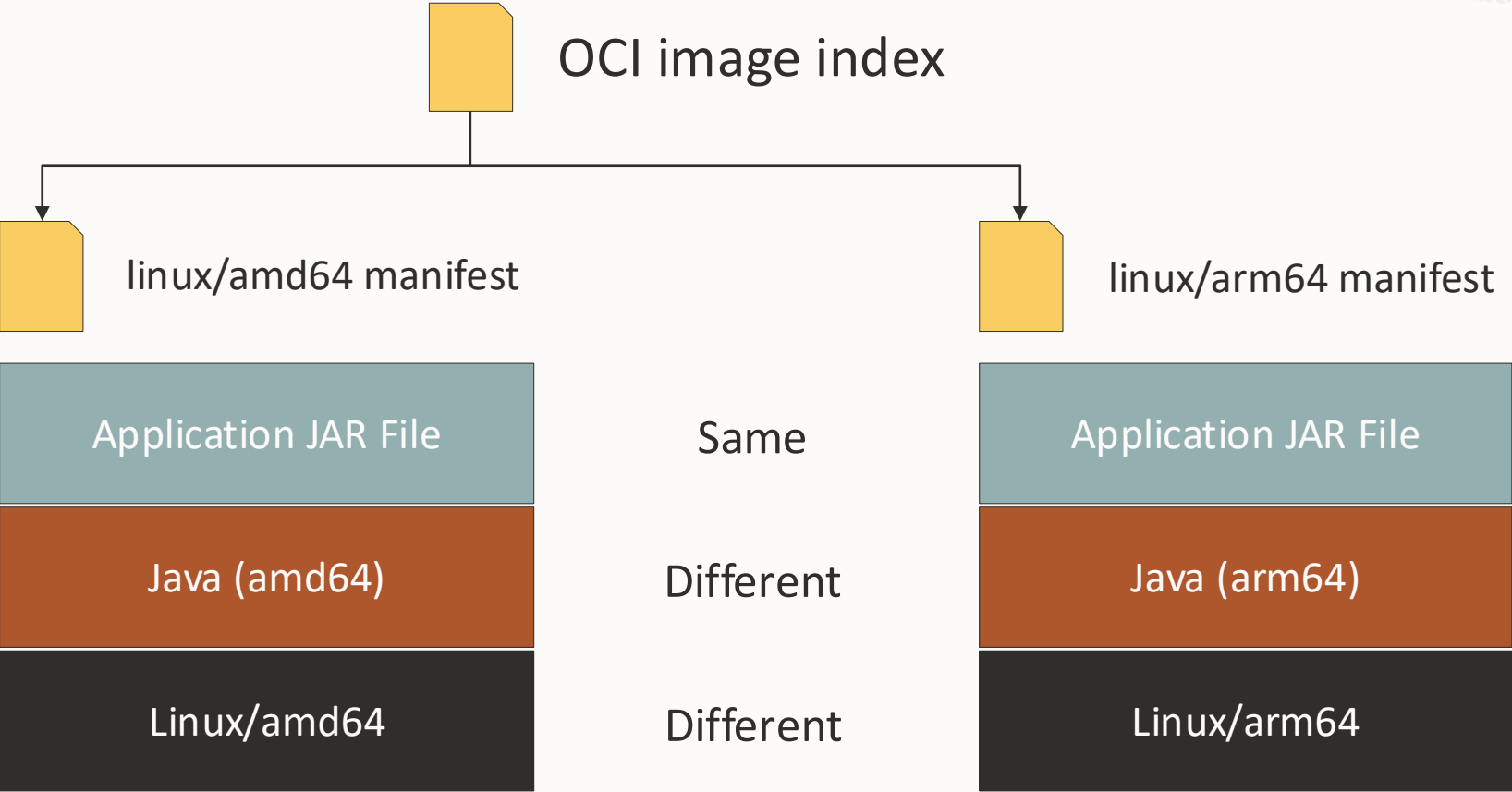
- A multi-architecture image is an Open Container Initiative (OCI) image index, not one portable native image.
- The runtime selects the manifest matching its OS and CPU architecture.

Multi-Architecture Container Images



- The application JAR file is portable bytecode; the Java runtime and native-derived artefacts are platform-specific.

Multi-Architecture Container Images



Multi-Architecture Container Images

- Specify the target platforms when building the container image

```
> docker buildx build \
  --platform 'linux/amd64,linux/arm64' \
  --file './containers/Dockerfile' \
  --tag 'demo:application-oci' \
  --output 'type=oci,dest=./target/application.tar' \
  .
```

Multi-Architecture Container Images



- Developers, CI and production machines use different architectures
- Production uses ARM instances for cost or energy efficiency

Demo

multi-architecture-image-builds-
spring-boot

Next



Proof of concept

- **Identify** an application
- **Experiment** and compare the results



How can we help

- Ad hoc conversations
- Workshops
- Knowledge Transfer
- Pairing sessions
- Hands-on



We are here to help

You are not alone
Oracle is here to help



Thank you

Albert Attard

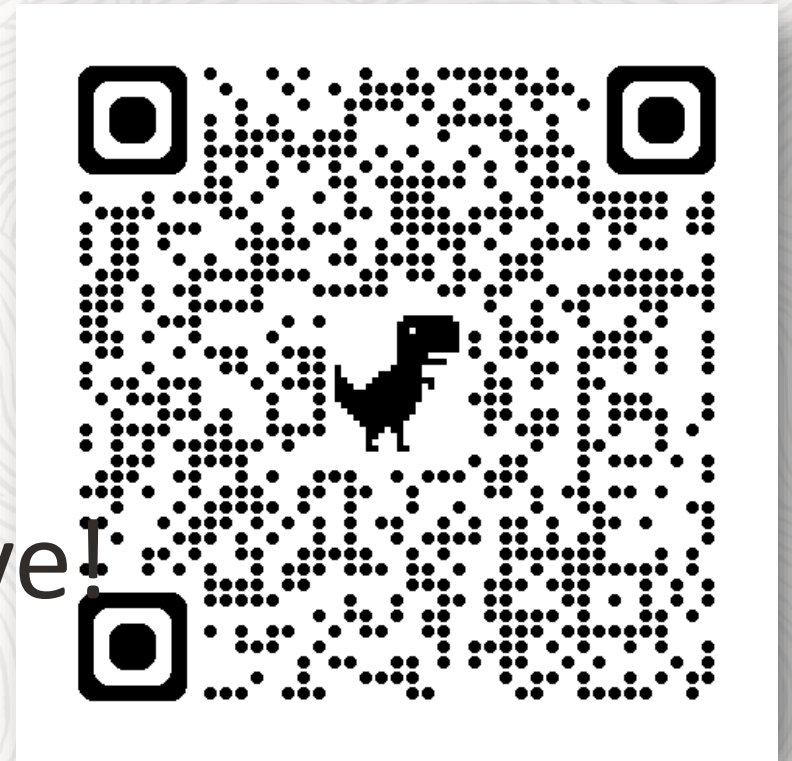
Principal Java Success Consultant

albert.attard@oracle.com



Your feedback is invaluable to us!
Please share your thoughts
through our [brief survey](#) or
by email:
albert.attard@oracle.com.

Thank you for helping us improve!



ORACLE

Our mission is to help people see
data in new ways, discover insights,
unlock endless possibilities.

